

THE COMPLETE GUIDE TO AI ENGINEERING

The **AI** Engineer's Complete Handbook

From your first API call to production AI systems
trusted by thousands of users.

⚡ Beginner

🔗 Intermediate

🚀 Advanced

30

CHAPTERS

3

LEVELS

50+

CODE EXAMPLES

∞

VALUE

Written by **Rohith Shabad** • AI Engineer & Educator

Contents

PART ONE — BEGINNER

- Ch. 1 What Is AI? A Complete Beginner's Guide
- Ch. 2 Your First ChatGPT App in 10 Minutes
- Ch. 3 Prompt Engineering 101: Talk to AI Like a Pro
- Ch. 4 Building a PDF Q&A Bot (Your First RAG App)
- Ch. 5 AI Tools Every Developer Should Know in 2025
- Ch. 6 Build a Customer Support AI Bot
- Ch. 7 AI for Productivity: Automate Your Work with AI
- Ch. 8 Deploy Your AI App to the World

PART TWO — INTERMEDIATE

- Ch. 9 Building Production Apps with LangChain
- Ch. 10 Vector Databases & Semantic Search at Scale
- Ch. 11 Production RAG: From Prototype to Scale
- Ch. 12 Building AI Agents with Tools
- Ch. 13 Prompt Engineering Mastery
- Ch. 14 AI APIs: OpenAI, Anthropic & Choosing the Right Model
- Ch. 15 Streaming, Async & Production Performance
- Ch. 16 Full-Stack AI Application: End to End

PART THREE — ADVANCED

- Ch. 17 Advanced RAG: Hybrid Search, Re-Ranking & Query Rewriting
- Ch. 18 Fine-Tuning with LoRA: Domain-Specific Models

Ch. 19	LangGraph: Stateful Agent Workflows
Ch. 20	Production AI Evaluation
Ch. 21	Multimodal AI: Vision, PDFs & Screenshot Agents
Ch. 22	Shipping AI SaaS Products
Ch. 23	Self-Hosted LLMs with Ollama & vLLM
Ch. 24	The AI Architect's Playbook
Ch. 25	DSPy: Programmatic Prompt Optimization
Ch. 26	GraphRAG: Knowledge Graphs for Complex Reasoning
Ch. 27	AI Memory Systems: Episodic, Semantic & Procedural
Ch. 28	MCP: Model Context Protocol
Ch. 29	Build a Cursor AI Clone
Ch. 30	CrewAI vs LangGraph vs AutoGen

Foreword

Why this book exists

I got into AI engineering the hard way — reading papers I barely understood, copying code from tutorials that didn't work, and spending weekends debugging APIs that changed faster than the documentation.

Three years later, I've built AI systems used by real companies, shipped products with real paying customers, and taught thousands of engineers how to do the same. And I realized: there's no single resource that takes you from "what is ChatGPT" all the way to "I just shipped a production AI system that earns money."

This book is that resource. It's organized as three acts — Beginner (concepts and your first apps), Intermediate (frameworks, RAG, and agents), and Advanced (production systems, fine-tuning, and everything senior engineers actually use). You can read it cover to cover or jump to the level that matches where you are today.

"The best time to learn AI engineering was 3 years ago. The second best time is right now — before the gap between those who know and those who don't gets any wider."

Every concept is paired with working code. Every chapter ends with key learnings. Every section is written for engineers, by an engineer — no fluff, no padding, just the knowledge you need to build things that work.

Let's build something great.



Rohith Shabad

AI Engineer & Course Creator

<https://ai-stack-picker-nine.vercel.app>

PART ONE

Beginner

Start from zero. Build your first real AI apps. No prior experience required. By the end of Part One, you'll have built a chatbot, a RAG Q&A system, a customer support bot, and deployed a live app.

8 Chapters

What Is AI? A Complete Beginner's Guide

Artificial intelligence is no longer science fiction. It's the autocomplete on your phone, the face unlock on your laptop, the recommendation on Netflix. But what exactly is it — and how does it work under the hood?

What AI Actually Is

Artificial Intelligence is software that learns patterns from data instead of following hand-written rules. Traditional software says: if the temperature is above 30°C, turn on the AC. AI software says: I've seen 10 million homes — here's what pattern of sensor readings usually means people are uncomfortable.

Machine Learning is the most common type of AI today. You show it thousands of examples, and it figures out the rules itself. Deep Learning is a subset that uses neural networks — layers of math inspired loosely by how neurons fire in the brain.

How LLMs Like ChatGPT Work

Large Language Models (LLMs) are trained on massive amounts of text — books, websites, code, papers. During training, the model learns to predict: given these words, what word comes next? Do this billions of times and something remarkable emerges — the model starts to understand grammar, facts, reasoning, and even coding.

When you chat with ChatGPT, you're giving it a sequence of tokens (words/parts of words) and asking it to predict the most likely continuation. That's it. The magic is in having seen so much human writing that the predictions become incredibly useful.

Real-World AI You Already Use

AI is not just chatbots. It's everywhere: Gmail's Smart Reply, Spotify's Discover Weekly, fraud detection on your credit card, Google Maps rerouting around traffic, the camera on your iPhone that makes photos look great. You've been using AI products for years — now it's time to build them.

```
# Your first AI call – 5 lines of Python from openai
import OpenAI client = OpenAI() # reads OPENAI_API_KEY
from environment response =
client.chat.completions.create( model="gpt-4o-mini",
messages=[{"role": "user", "content": "Explain AI in
one sentence"}] )
print(response.choices[0].message.content)
```

PRO TIP

Install Python 3.10+, run `pip install openai`, set your API key as an environment variable, and run this script. You're now an AI developer.

KEY LEARNINGS

- ✓ AI learns patterns from data instead of following rigid rules
- ✓ LLMs predict the next token based on massive training data
- ✓ Machine learning, deep learning, and neural networks are related concepts
- ✓ AI is already embedded in apps you use every day
- ✓ You don't need a math PhD to build AI applications

Your First ChatGPT App in 10 Minutes

Theory is nice. Working code is better. In this chapter we build a real chatbot from scratch — one that maintains conversation history, streams responses in real time, and runs on your laptop.

Setting Up Your Environment

You need three things: Python 3.10+, the OpenAI SDK, and an API key. Get your key at platform.openai.com. Store it as an environment variable — never hardcode secrets in your code.

Understanding Tokens and Cost

OpenAI charges per token — roughly 4 characters. GPT-4o mini costs \$0.15 per million input tokens. A typical conversation is ~500 tokens. You can have thousands of conversations for a dollar. Use the Tokenizer tool at platform.openai.com/tokenizer to visualize.

Building a Conversation Loop

A chatbot needs memory. The OpenAI API is stateless — every call starts fresh. To give your bot memory, you maintain a list of messages and send the entire history with each request. This is how ChatGPT itself works.

```
import os from openai import OpenAI client = OpenAI()
history = [{"role": "system", "content": "You are a
helpful assistant."}] while True: user_input =
input("You: ") history.append({"role": "user",
"content": user_input}) response =
client.chat.completions.create( model="gpt-4o-mini",
messages=history, stream=True ) print("AI: ", end="",
flush=True) reply = "" for chunk in response: if
chunk.choices[0].delta.content: text =
chunk.choices[0].delta.content print(text, end="",
flush=True) reply += text print()
history.append({"role": "assistant", "content": reply})
```

PRO TIP

Add a system message at the start of your history list to set the AI's personality and behavior. This is the most powerful single prompt technique.

KEY LEARNINGS

- ✓ Set API keys as environment variables, never in code
- ✓ The messages list IS the conversation memory — send the full history
- ✓ Tokens are ~4 characters — understand costs before scaling
- ✓ Stream responses with stream=True for a much better UX
- ✓ gpt-4o-mini is great for development — cheap and fast

Prompt Engineering 101: Talk to AI Like a Pro

The difference between a useless AI response and an amazing one is often just how you ask. Prompt engineering is the skill of communicating effectively with language models — and it's learnable in an afternoon.

Why Prompts Matter So Much

LLMs are completion machines — they predict what text naturally follows yours. If your prompt is vague, the model fills the gap with its best guess, which may not match your intent. Specific prompts get specific results.

The Core Techniques

Zero-shot: Just ask directly. Works for simple tasks.

Few-shot: Show 2-3 examples of what you want. Dramatically improves quality on structured tasks.

Chain-of-thought: Add 'Think step by step' and the model reasons through problems instead of guessing the answer.

Role prompting: 'You are a senior Python engineer reviewing code for security vulnerabilities' focuses the model's 'expertise'.

Structured Output

Need JSON, not prose? Tell the model exactly what format you want — and use `response_format` with `json_object` mode in the API for guaranteed valid JSON. Never parse free-form text when you can get structured data.

```
# Few-shot prompting example
prompt = """ Classify the sentiment of each review as positive, negative, or neutral. Examples: Review: "Amazing product, exceeded expectations!" → positive Review: "Broken on arrival, very disappointed" → negative Review: "It arrived on time" → neutral Now classify this: Review: "Works okay but a bit slow" """
# Add "Think step by step" for reasoning tasks:
reasoning_prompt = "Solve this math problem. Think step by step.\n\n" + problem
```

PRO TIP

Test every prompt change with at least 5-10 different inputs before deciding it's an improvement. One good example can mask a prompt that fails on edge cases.

KEY LEARNINGS

- ✓ Be specific — vague prompts get vague answers
- ✓ Few-shot examples dramatically improve output quality
- ✓ Chain-of-thought ('think step by step') unlocks reasoning
- ✓ Role prompting focuses the model on your domain
- ✓ Use `json_object` mode for reliable structured output

Building a PDF Q&A Bot (Your First RAG App)

What if your AI could read your documents — a product manual, a legal contract, a research paper — and answer questions about them accurately? That's RAG: Retrieval Augmented Generation. Let's build one.

What Is RAG and Why Does It Exist

LLMs have a knowledge cutoff and can't access your private documents. RAG solves this by: (1) breaking your documents into chunks, (2) converting chunks to vectors that capture meaning, (3) when a question arrives, finding the most relevant chunks, (4) giving those chunks to the LLM as context. The LLM answers from your documents, not from its training data.

Embeddings: Numbers That Capture Meaning

An embedding is a list of ~1500 numbers that represents the meaning of a piece of text. Similar meanings are close in this high-dimensional space. 'dog' and 'puppy' are close. 'dog' and 'mortgage' are far apart. This is how semantic search works.

Building the Pipeline

Load the PDF → split into 500-token chunks with 50-token overlap
→ embed each chunk → store in a vector database → on query:

embed the question → find top-5 similar chunks → build a prompt with those chunks → get the LLM to answer.

```
from langchain_community.document_loaders import
PyPDFLoader from langchain.text_splitter import
RecursiveCharacterTextSplitter from langchain_openai
import OpenAIEmbeddings, ChatOpenAI from
langchain_community.vectorstores import Chroma from
langchain.chains import RetrievalQA # Load and split
loader = PyPDFLoader("your_document.pdf") docs =
loader.load() splitter =
RecursiveCharacterTextSplitter(chunk_size=500,
chunk_overlap=50) chunks =
splitter.split_documents(docs) # Embed and store
vectorstore = Chroma.from_documents(chunks,
OpenAIEmbeddings()) # Ask questions qa =
RetrievalQA.from_chain_type( llm=ChatOpenAI(model="gpt-4o-
mini"),
retriever=vectorstore.as_retriever(search_kwargs={"k":
5}) ) answer = qa.invoke("What are the key terms in
section 3?")
```

PRO TIP

Always add 'Only answer based on the provided context. If the answer is not in the context, say you don't know.' to prevent hallucination.

KEY LEARNINGS

- ✓ RAG connects LLMs to your private documents
- ✓ Embeddings are vectors that capture semantic meaning
- ✓ Chunk size matters — 500 tokens with overlap works well
- ✓ Retrieve top-k chunks, not just the best one
- ✓ Always cite sources so users can verify answers

AI Tools Every Developer Should Know in 2025

The AI tools landscape changes every month. This chapter cuts through the noise — here are the tools that matter, why they exist, and when to reach for each one.

The Framework Layer

LangChain: The most popular framework. Great for RAG, chains, and getting started fast. Can be verbose.

LlamaIndex: Purpose-built for data and RAG. More efficient for document-heavy apps. Better for production RAG.

DSPy: Automatically optimizes prompts using data. Use when you need programmatic, reliable prompting at scale.

The Model Layer

OpenAI GPT-4o: Best reasoning, great for complex tasks. Costs more.

Claude 3.5 Sonnet: Excellent for long documents and coding. 200K context window.

Gemini Pro: Best for multimodal. Google ecosystem.

Mistral/Llama 3: Open-source. Self-host for free. Great for fine-tuning.

The Data Layer

Pinecone: Managed vector DB. Easiest to use in production.

Weaviate: Open-source, self-hostable, powerful.

ChromaDB: Local vector DB. Perfect for prototyping.

pgvector: Add vectors to your existing PostgreSQL. Often the right choice.

```
# Switching between providers with one line change from
langchain_openai import ChatOpenAI from
langchain_anthropic import ChatAnthropic # Just swap
the model – same interface llm =
ChatOpenAI(model="gpt-4o-mini") # OpenAI # llm =
ChatAnthropic(model="claude-3-5-sonnet-latest") #
Anthropic response = llm.invoke("Explain embeddings in
2 sentences") print(response.content)
```

PRO TIP

Abstract your LLM behind an interface so you can swap providers without rewriting your app. LangChain's ChatModel interface does this for you.

KEY LEARNINGS

- ✓ LangChain for getting started, LlamaIndex for production RAG
- ✓ GPT-4o for quality, Claude for long docs, Mistral for self-hosting
- ✓ ChromaDB for local dev, Pinecone for production
- ✓ Start with the simplest tool that solves your problem
- ✓ Don't lock yourself into one provider — abstract the LLM layer

Build a Customer Support AI Bot

Customer support is one of the highest-ROI applications of AI. In this chapter we build a real support bot — one that handles FAQs from your knowledge base, knows when to escalate, and improves over time.

Architecture of a Support Bot

A good support bot has: an FAQ knowledge base (your product docs as RAG), intent detection (is this a question, a complaint, a refund request?), a routing layer (simple Q: answer with RAG, complex: escalate to human), and conversation history to avoid repeating information.

Building the Knowledge Base

Collect all your FAQs, product docs, and support ticket resolutions. Load them into a vector store. Add metadata (category, product, date) so you can filter: 'only search the billing docs for billing questions'.

Handling Escalation

Build an escalation trigger: if the bot's confidence is low, if the user expresses frustration, or if the query matches patterns like 'refund', 'broken', 'cancel' — route to a human. Never let AI handle things it shouldn't.

```
def support_bot_response(user_message, history,
vectorstore): # Check for escalation triggers
escalation_keywords = ["refund", "cancel", "broken",
"lawsuit", "manager"] if any(kw in user_message.lower()
for kw in escalation_keywords): return "I'm connecting
you with a human agent right away.", True # Retrieve
relevant context docs =
vectorstore.similarity_search(user_message, k=3)
context = "\n".join([d.page_content for d in docs]) #
Build prompt with context messages = [ {"role":
"system", "content": f"You are a helpful support agent.
\n\nContext:\n{context}"}, *history, {"role": "user",
"content": user_message} ] response =
client.chat.completions.create(model="gpt-4o-mini",
messages=messages) return
response.choices[0].message.content, False # False = no
escalation
```

PRO TIP

Add a 'confidence score' by asking the LLM to rate its own certainty (1-5). If below 3, escalate. LLMs are surprisingly good at knowing when they don't know.

KEY LEARNINGS

- ✓ Structure your knowledge base with categories for filtered retrieval
- ✓ Always build an escalation path to humans
- ✓ Detect sentiment to route frustrated users immediately
- ✓ Log all conversations for review and improvement
- ✓ Measure resolution rate — not just response time

AI for Productivity: Automate Your Work with AI

AI's biggest near-term impact isn't dramatic AGI — it's saving you 2 hours every day on repetitive knowledge work. Let's build automations that actually save time.

Email Summarization Pipeline

If you have a long email thread, AI can extract: the key decision made, who needs to do what, and the deadline. Build this as a simple function that takes email content and returns a structured summary. Run it every morning on your inbox.

Document Processing Automation

Need to extract information from 50 PDFs? Invoices, contracts, resumes? AI can read each one, extract structured fields (name, date, amount, parties), and output clean data. What would take hours takes seconds.

Meeting Notes to Action Items

Record your meeting, transcribe with Whisper, then ask GPT to extract: meeting summary, decisions made, action items with owners and deadlines, and follow-up questions. Better than any note-taker.

```
import asyncio from openai import AsyncOpenAI client =
AsyncOpenAI() async def summarize_email(email_text:
str) → dict: response = await
client.chat.completions.create( model="gpt-4o-mini",
messages=[{ "role": "user", "content": f"""Analyze this
email and respond with JSON: {{"summary": "...",
"action_items": [ ... ], "deadline": "..."}" } Email:
{email_text}"""} ], response_format={"type":
"json_object"} ) return
json.loads(response.choices[0].message.content) #
Process 50 emails concurrently results = await
asyncio.gather(*[summarize_email(e) for e in emails])
```

PRO TIP

Use batch processing with `asyncio.gather()` to process multiple documents simultaneously. You can process 50 documents in the time it takes to process 1 sequentially.

KEY LEARNINGS

- ✓ Email summarization: extract decisions, actions, deadlines
- ✓ Document extraction: turn PDFs into structured JSON
- ✓ Whisper + GPT = perfect meeting notes automatically
- ✓ Batch process with async for 10x throughput
- ✓ Always have a human review automated outputs initially

Deploy Your AI App to the World

You've built something. Now let's make it live.

Deployment is the step most beginners skip — and then wonder why no one uses their app. This chapter gets you from localhost to live URL in one hour.

Packaging with FastAPI

FastAPI is the fastest way to turn your Python AI code into a web API. It's async-first, auto-generates API docs, and deploys everywhere. Wrap your chatbot in a FastAPI endpoint and you have a production-ready backend.

Frontend with Streamlit

Streamlit turns Python scripts into web apps with almost no extra code. Add `st.chat_input()` and `st.chat_message()` and you have a ChatGPT-style interface. Deploy to Streamlit Cloud for free.

Deploying to Railway

Railway is the easiest cloud platform for AI apps. Connect your GitHub repo, add your environment variables (including `OPENAI_API_KEY`), and Railway auto-deploys on every push. Free tier for small apps.

```
# app.py – FastAPI chatbot backend from fastapi import
FastAPI from pydantic import BaseModel from openai
import OpenAI app = FastAPI() client = OpenAI() class
ChatRequest(BaseModel): message: str history: list = []
@app.post("/chat") async def chat(req: ChatRequest):
messages = req.history + [{"role": "user", "content":
req.message}] response =
client.chat.completions.create( model="gpt-4o-mini",
messages=messages ) return {"reply":
response.choices[0].message.content} # Run: uvicorn
app:app --reload # Deploy: push to GitHub, connect to
Railway
```

PRO TIP

Add a Procfile with `web: uvicorn app:app --host 0.0.0.0 --port \$PORT` and Railway will auto-detect your deployment configuration.

KEY LEARNINGS

- ✓ FastAPI wraps your Python logic in a production web API instantly
- ✓ Streamlit creates chat UIs in 10 lines of Python
- ✓ Never put API keys in your code — use environment variables
- ✓ Railway and Render deploy from GitHub automatically
- ✓ Add a /health endpoint so you know your app is running

PART TWO

Intermediate

Go from "it works on my machine" to production-grade AI applications. You'll master LangChain, vector databases, production RAG, AI agents, and build a complete full-stack AI product.

8 Chapters

Building Production Apps with LangChain

LangChain is the most popular AI framework — but most tutorials only scratch the surface. This chapter goes deep on the patterns senior engineers actually use in production.

LangChain Expression Language (LCEL)

LCEL uses the pipe operator `|` to chain components: `prompt | llm | parser`. This creates declarative, composable pipelines that are easy to test, modify, and deploy. It also handles async, streaming, and batching automatically.

Runnable Patterns

Every LangChain component implements the `Runnable` interface with `invoke()`, `stream()`, `batch()`, and `ainvoke()`. This uniformity means you can swap out any component — change the LLM, the parser, the retriever — without touching the rest of the chain.

Error Handling and Fallbacks

Production chains need fallbacks. Use `.with_fallbacks([backup_llm])` to automatically retry with a different model if the primary fails. Add `.with_retry(stop_after_attempt=3)` for transient errors. Never let a single API failure crash your whole app.

```

from langchain_openai import ChatOpenAI from
langchain_core.prompts import ChatPromptTemplate from
langchain_core.output_parsers import StrOutputParser #
Build a chain with LCEL prompt =
ChatPromptTemplate.from_template("Summarize this in 3
bullets:\n\n{text}") llm = ChatOpenAI(model="gpt-4o-
mini") parser = StrOutputParser() chain = prompt | llm
| parser # That's it! # Add fallback for resilience
backup_llm = ChatOpenAI(model="gpt-3.5-turbo")
resilient_chain = prompt |
llm.with_fallbacks([backup_llm]) | parser # Stream
responses for chunk in resilient_chain.stream({"text":
long_article}): print(chunk, end="", flush=True)

```

PRO TIP

Use LangSmith for tracing your chains in production. Every LLM call, retrieval, and tool use is logged with input/output and latency. Essential for debugging.

KEY LEARNINGS

- ✓ LCEL pipe syntax creates clean, composable chains
- ✓ Every component is a Runnable — swap anything without refactoring
- ✓ Use `.with_fallbacks()` for resilient production chains
- ✓ Async and streaming are first-class in LCEL
- ✓ Test chains component by component, then end-to-end

Vector Databases & Semantic Search at Scale

Vector databases are the memory layer of AI applications. In this chapter we go beyond ChromaDB on localhost — we build production semantic search that handles millions of documents.

How Vector Search Works

When you search for 'dog food recipes', a keyword search finds pages with those exact words. Semantic search finds pages about 'pet nutrition guides' and 'canine diet tips' — because the embeddings are nearby in vector space. This is fundamentally different and far more useful for AI applications.

Pinecone in Production

Pinecone is a fully managed vector database. Create an index, upsert your vectors with metadata, and query. It handles sharding, replication, and scaling automatically. Use metadata filtering to narrow searches: filter by department, date range, or product category before the vector search runs.

Hybrid Search: The Best of Both Worlds

Pure vector search misses exact matches. Pure keyword search misses semantic meaning. Hybrid search combines both: use BM25 for keyword matching, vector search for semantic similarity, then

merge the results with Reciprocal Rank Fusion (RRF). This reliably outperforms either approach alone.

```
import pinecone from pinecone import Pinecone,
ServerlessSpec pc =
Pinecone(api_key=os.getenv("PINECONE_API_KEY")) #
Create index pc.create_index( name="documents",
dimension=1536, # OpenAI embedding size
metric="cosine", spec=ServerlessSpec(cloud="aws",
region="us-east-1") ) index = pc.Index("documents") #
Upsert with metadata for filtering
index.upsert(vectors=[ {"id": "doc_1", "values":
embedding, "metadata": {"category": "billing", "date":
"2025-01"}} ] ) # Query with filter results =
index.query( vector=query_embedding, top_k=5,
filter={"category": {"$eq": "billing"}} )
```

PRO TIP

Always benchmark your retrieval before building on top of it. Ask 20 representative questions, check if the right documents come back in the top 5. Fix retrieval before fixing the LLM.

KEY LEARNINGS

- ✓ Semantic search finds meaning, not just keywords
- ✓ Always add metadata to vectors for filtered retrieval
- ✓ Hybrid search (BM25 + vectors) outperforms either alone
- ✓ Pinecone handles production scaling — ChromaDB is for local only
- ✓ Namespace your vectors by tenant for multi-tenant apps

Production RAG: From Prototype to Scale

Getting RAG to work in a demo is easy. Getting it to work reliably in production — with real users, diverse questions, and a need to prove it's accurate — is the real challenge. This chapter covers the techniques that actually matter.

Query Rewriting

Users ask terrible questions. 'The thing from last week' or 'that error again' are not searchable. Query rewriting uses the LLM to reformulate the query before retrieval: expand abbreviations, resolve pronouns, and generate 3 alternative phrasings. Retrieve for all 4 and merge results.

Contextual Chunking

The chunk that contains the answer to a question often lacks context. 'The revenue increased by 15%' — what product? What year? Use parent document retrieval: chunk at 200 tokens for search precision, but retrieve the 1000-token parent for full context.

Evaluating RAG with RAGAS

RAGAS measures: Faithfulness (did the answer come from the context?), Answer Relevance (did it answer the question?), Context Precision (were the retrieved chunks relevant?), and Context Recall

(were all relevant chunks retrieved?). Run these metrics on a test set automatically.

```
from ragas import evaluate from ragas.metrics import
faithfulness, answer_relevancy, context_precision #
Build your test dataset test_data = [ {"question":
"What is the refund policy?", "ground_truth": "30-day
full refund", "contexts": retrieved_chunks, "answer":
generated_answer} ] from datasets import Dataset
dataset = Dataset.from_list(test_data) # Evaluate
automatically results = evaluate( dataset,
metrics=[faithfulness, answer_relevancy,
context_precision] ) print(results) # {"faithfulness":
0.92, "answer_relevancy": 0.87, ... }
```

PRO TIP

Build your evaluation dataset before building your RAG pipeline. 50 question-answer pairs from real users is worth more than 1000 synthetic ones.

KEY LEARNINGS

- ✓ Rewrite queries before retrieval — users ask bad questions
- ✓ Parent document retrieval: small chunks for search, big for context
- ✓ RAGAS gives you automated, objective RAG evaluation metrics
- ✓ Re-ranking (Cohere) dramatically improves retrieval quality
- ✓ Test with diverse question types: factual, comparative, multi-hop

Building AI Agents with Tools

An agent is an LLM that can decide what to do, call tools to take action, observe the result, and repeat until the task is done. This chapter builds agents that actually work on real tasks.

The ReAct Loop

ReAct agents follow: Reason (what do I need to do?), Act (call a tool), Observe (what did the tool return?), Repeat until done. The key insight is that the LLM's output can be an action specification, not just text.

Tool Definition and Function Calling

OpenAI function calling lets you define tools as JSON schemas. The model decides when to call them and with what parameters. The result comes back to the model, which incorporates it into its reasoning. Build tools for: web search, code execution, database queries, API calls.

When Agents Go Wrong

Agents can loop forever, call tools unnecessarily, or confidently make wrong decisions. Add: maximum iterations limit, tool call logging, human approval for destructive actions (delete, send, purchase), and structured output to prevent malformed tool calls.

```

from langchain.agents import create_tool_calling_agent,
AgentExecutor from langchain_core.tools import tool
from langchain_openai import ChatOpenAI @tool def
search_web(query: str) → str: """Search the web for
current information about a topic.""" # Implement with
SerpAPI, Tavily, or DuckDuckGo return search_results
@tool def run_python(code: str) → str: """Execute
Python code and return the output.""" # Use a sandboxed
executor in production! return exec_result llm =
ChatOpenAI(model="gpt-4o", temperature=0) tools =
[search_web, run_python] agent =
create_tool_calling_agent(llm, tools, prompt) executor
= AgentExecutor(agent=agent, tools=tools,
max_iterations=10, verbose=True) result =
executor.invoke({"input": "What is the weather in
Austin, TX right now?"})

```

PRO TIP

Use temperature=0 for agents. Agents need to be deterministic and predictable, not creative. Save creativity for the final response generation step.

KEY LEARNINGS

- ✓ ReAct: Reason → Act → Observe → Repeat until done
- ✓ Define tools as JSON schemas with clear descriptions
- ✓ Always limit max iterations — agents can loop forever
- ✓ Log every tool call for debugging and audit
- ✓ Human-in-the-loop for any irreversible action

Prompt Engineering Mastery

You've learned the basics. This chapter covers advanced techniques used by AI engineers at top labs — structured prompting, constitutional AI patterns, and prompt version control.

Structured Output Engineering

Don't ask for JSON and hope. Use Pydantic models with instructor library to guarantee type-safe, validated structured output. If the model returns bad JSON, instructor retries automatically. Use this for any extraction or classification task.

Meta-Prompting

Ask the LLM to improve its own prompt. Give it your current prompt, sample failures, and ask it to rewrite the prompt to avoid those failures. Iterate 3-5 times. This often finds prompt improvements a human would never think of.

Prompt Version Control

Treat prompts like code. Store them in version control. Use a tool like LangSmith or PromptLayer to track which prompt version is in production, run A/B tests, and roll back when a new prompt performs worse.

```
import instructor from openai import OpenAI from
pydantic import BaseModel from typing import List
```

```
client = instructor.from_openai(OpenAI()) class
ExtractedData(BaseModel): company_name: str revenue:
float key_risks: List[str] recommendation: str #
Guaranteed type-safe output – retries automatically if
malformed result =
client.chat.completions.create( model="gpt-4o",
response_model=ExtractedData, messages=[{"role":
"user", "content": f"Extract from this report:
\n{report_text}"}] ) print(result.company_name) #
Typed, validated Python object
```

PRO TIP

The instructor library is one of the best investments you can make. It turns 'AI returns messy JSON sometimes' into 'AI returns typed Python objects always'.

KEY LEARNINGS

- ✓ Use Pydantic + instructor for guaranteed structured output
- ✓ Meta-prompting: let the LLM improve its own prompts
- ✓ Version control your prompts like code — never edit in production directly
- ✓ A/B test prompt changes before deploying
- ✓ Few-shot examples in production should be curated, not random

AI APIs: OpenAI, Anthropic & Choosing the Right Model

Not all LLMs are equal. Choosing the wrong model for your task costs money and quality. This chapter gives you a framework for picking the right model every time.

The Model Selection Framework

Ask these questions: How complex is the reasoning? (complex = GPT-4o, simple = mini), How long is the context? (long = Claude with 200K), Do I need guaranteed JSON? (any modern model with tools), Is privacy critical? (self-hosted Llama), How tight is the cost budget? (mini models or self-hosted).

Function Calling Deep Dive

Function calling (now called 'tool use') lets models decide when and how to call your functions. Parallel tool calling (multiple tools at once) dramatically speeds up agents. Strict mode guarantees the output exactly matches your schema — no more parsing errors.

Cost Optimization

Cache your system prompts with Anthropic's prompt caching (90% cost reduction on repeated system prompts). Use gpt-4o-mini for classification and extraction, gpt-4o only for complex reasoning. Batch API for async processing at 50% cost. Monitor spend daily.

```
# Parallel tool calling – two searches at once response
= client.chat.completions.create( model="gpt-4o",
tools=tools, tool_choice="auto", messages=messages ) #
Process multiple tool calls in parallel import asyncio
tool_calls = response.choices[0].message.tool_calls
results = await
asyncio.gather(*[ execute_tool(tc.function.name,
tc.function.arguments) for tc in tool_calls ]) #
Anthropic prompt caching (90% cost reduction) import
anthropic client = anthropic.Anthropic() response =
client.messages.create( model="claude-3-5-sonnet-
latest", system=[{"type": "text", "text":
long_system_prompt, "cache_control": {"type":
"ephemeral"}}], # Cache this! messages=messages )
```

PRO TIP

Set up billing alerts at \$10, \$50, and \$100 on every AI platform you use. One runaway script can generate a \$500 bill overnight. Learn from others' mistakes.

KEY LEARNINGS

- ✓ GPT-4o for reasoning, mini for extraction/classification, Claude for long docs
- ✓ Use parallel tool calling for faster agents
- ✓ Prompt caching cuts costs by 90% for repeated system prompts
- ✓ Batch API for async processing at half the price
- ✓ Always track token usage and set spend alerts

Streaming, Async & Production Performance

The difference between an AI app that feels snappy and one that feels slow is almost always streaming and async. This chapter covers the engineering patterns that make AI applications feel fast.

Streaming Responses

Instead of waiting 8 seconds for the full response, stream tokens as they're generated. Users see output immediately and perceive the app as much faster. Use Server-Sent Events (SSE) to stream from your FastAPI backend to your React frontend.

Async Everything

LLM calls are I/O bound — perfect for async. With `asyncio`, you can handle 100 concurrent users on a single server that would otherwise handle 5. Use async client libraries, async database queries, and async cache checks.

Caching for Cost and Speed

Cache identical queries with Redis. If 100 users ask 'what is your refund policy', make 1 LLM call and cache the response for 1 hour. Add semantic caching: if the new question is 95% similar to a cached one, return the cached answer. GPTCache does this automatically.

```
from fastapi import FastAPI from fastapi.responses
import StreamingResponse from openai import AsyncOpenAI
app = FastAPI() client = AsyncOpenAI() @app.post("/
chat/stream") async def stream_chat(message: str):
    async def generate(): stream = await
    client.chat.completions.create( model="gpt-4o-mini",
    messages=[{"role": "user", "content": message}],
    stream=True ) async for chunk in stream: if
    chunk.choices[0].delta.content: yield f"data:
    {chunk.choices[0].delta.content}\n\n" yield "data:
    [DONE]\n\n" return StreamingResponse(generate(),
    media_type="text/event-stream")
```

PRO TIP

Add a timeout parameter to every LLM call. A hung API call with no timeout will tie up your async worker indefinitely. `timeout=30.0` is a good default.

KEY LEARNINGS

- ✓ Stream responses — users tolerate waiting less when they see progress
- ✓ Async turns 5 concurrent users into 100+ on the same hardware
- ✓ Exact caching for identical queries, semantic caching for similar ones
- ✓ Use connection pooling for database calls inside async handlers
- ✓ Add request timeouts — never let an LLM call hang forever

Full-Stack AI Application: End to End

This is the capstone chapter for the intermediate course. We build a complete, deployed, production-quality AI application — a smart document analyzer with a beautiful UI, streaming, user authentication, and monitoring.

Architecture Overview

Backend: FastAPI + LangChain + Pinecone for RAG. Frontend: React + Tailwind with streaming support. Auth: Supabase for user management. Cache: Redis for semantic caching. Monitoring: LangSmith for traces + Sentry for errors. Database: PostgreSQL for user data.

The Frontend Streaming Pattern

React reads the SSE stream from FastAPI and appends tokens to the UI as they arrive. Use the Fetch API with ReadableStream or a library like react-query with manual SSE handling. The key: show the first token in under 500ms.

Observability and Monitoring

Every production AI app needs: LangSmith tracing (see every LLM call, latency, tokens used), error monitoring with Sentry, custom

metrics (resolution rate, user satisfaction, cost per conversation), and alerts when anything degrades.

```
# Frontend streaming in React async function*
streamResponse(message) { const response = await
fetch("/chat/stream", { method: "POST", headers:
{"Content-Type": "application/json"}, body:
JSON.stringify({ message }) }); const reader =
response.body.getReader(); const decoder = new
TextDecoder(); while (true) { const { done, value } =
await reader.read(); if (done) break; const text =
decoder.decode(value); const lines = text.split("\n");
for (const line of lines) { if (line.startsWith("data:
")) { const token = line.slice(6); if (token !==
"[DONE]") yield token; } } } } // Usage in
component: // for await (const token of
streamResponse(input)) { // setReply(prev => prev +
token); // }
```

PRO TIP

Build the full pipeline working end-to-end on day 1, even if individual components are basic. Then improve each layer iteratively. A running system beats a perfect design every time.

KEY LEARNINGS

- ✓ Full stack: FastAPI + React + Pinecone + Supabase + Redis
- ✓ Stream from backend to frontend with Server-Sent Events
- ✓ LangSmith for AI tracing, Sentry for error monitoring
- ✓ Track cost per conversation as a key business metric
- ✓ Blue-green deploy to update without downtime

PART THREE

Advanced

Production AI engineering at scale. Fine-tune your own models, build stateful agents, evaluate AI systematically, and architect systems that serve millions. This is what senior AI engineers actually know.

14 Chapters

Advanced RAG: Hybrid Search, Re-Ranking & Query Rewriting

Basic RAG gets you to 70% accuracy. Getting to 95% requires hybrid search, re-ranking, and smart query rewriting. This is the difference between a demo and a product.

Hybrid Search

BM25 finds exact keyword matches. Vector search finds semantic matches. Hybrid search uses Reciprocal Rank Fusion (RRF) to merge both ranked lists. It almost always outperforms either alone. Implement with Weaviate's hybrid parameter or manually merge results.

Cohere Re-Ranking

After retrieval, send your top-20 results to Cohere's re-ranker. It reads the full document and query and re-scores based on actual relevance. The re-ranker regularly promotes result #15 to #1. One API call, 40% better precision.

HyDE Query Expansion

Hypothetical Document Embeddings: ask the LLM to hallucinate what a perfect answer document would look like, then embed that hallucinated document and use it to search. This works because the

hallucinated answer is closer in embedding space to real answers than the original question.

```
from langchain_weaviate.vectorstores import
WeaviateVectorStore import cohere # Hybrid search in
Weaviate results =
vectorstore.similarity_search( query, k=20,
search_type="hybrid", alpha=0.5 # 0=BM25, 1=vector ) #
Re-rank with Cohere co =
cohere.Client(os.getenv("COHERE_API_KEY")) reranked =
co.rerank( query=query, documents=[r.page_content for r
in results], top_n=5, model="rerank-english-v3.0" )
top_docs = [results[r.index] for r in reranked.results]
```

PRO TIP

Hybrid search + re-ranking is your fastest path from 70% to 95% RAG accuracy. Implement both before anything else.

KEY LEARNINGS

- ✓ Hybrid search (BM25 + vectors) with RRF fusion
- ✓ Cohere re-ranking to reorder top-20 results
- ✓ HyDE: embed a hallucinated answer to search better
- ✓ Parent document retrieval for full context
- ✓ RAGAS evaluation: faithfulness, relevancy, precision, recall

Fine-Tuning with LoRA: Domain-Specific Models

When prompting isn't enough and you need a model that truly specializes in your domain, fine-tuning is the answer. LoRA makes it accessible — train on consumer hardware in hours.

How LoRA Works

Full fine-tuning updates all 7 billion parameters — expensive and slow. LoRA adds small trainable matrices (rank 8-64) alongside the frozen original weights. Only the adapters train. Result: 99% fewer parameters to train, same quality improvement, adapters are 50MB not 14GB.

Dataset Preparation

Quality beats quantity. 500 high-quality examples outperform 10,000 mediocre ones. Format: instruction-input-output triplets. Clean your data: remove duplicates, fix errors, ensure consistent formatting. Use GPT-4 to generate or clean training data — it's worth the cost.

Training and Evaluation

Use Hugging Face's TRL library with SFTTrainer. Monitor training loss and eval loss — if eval loss rises while training loss falls, you're

overfitting. Evaluate on a held-out test set with your actual use case metrics, not just perplexity.

```
from transformers import AutoModelForCausalLM,
AutoTokenizer from peft import LoraConfig,
get_peft_model from trl import SFTTrainer, SFTConfig
model =
AutoModelForCausalLM.from_pretrained("mistralai/
Mistral-7B-v0.1", load_in_4bit=True) lora_config =
LoraConfig( r=16, # Rank – higher = more capacity
lora_alpha=32, # Scale factor target_modules=["q_proj",
"v_proj"], lora_dropout=0.05, task_type="CAUSAL_LM" )
model = get_peft_model(model, lora_config) trainer =
SFTTrainer(model=model, train_dataset=dataset, ...)
trainer.train() model.save_pretrained("./my-fine-tuned-
model")
```

PRO TIP

Fine-tuning with LoRA: 4-bit base model, rank-16 adapter, 500+ quality examples, monitor eval loss, merge and deploy.

KEY LEARNINGS

- ✓ LoRA trains <1% of parameters with the same quality improvement
- ✓ 500 high-quality examples beat 10,000 mediocre ones
- ✓ Monitor eval loss — overfitting is the #1 failure mode
- ✓ Merge adapter weights for single-model deployment
- ✓ QLoRA: quantize base model to 4-bit for 24GB GPU training

LangGraph: Stateful Agent Workflows

LangGraph brings state machines to AI agents. Build agents that remember what they've done, branch based on results, and loop until the task is complete — without getting stuck.

Nodes and Edges

In LangGraph, your workflow is a directed graph. Nodes are functions that process state. Edges connect nodes — either direct (always go from A to B) or conditional (go to B or C based on state). This makes complex workflows visualizable and debuggable.

State Management

Every node reads from and writes to a shared state object. State is a TypedDict — define exactly what fields exist and their types. This prevents the 'agent randomly loses track of what it was doing' problem that plagues simple agent loops.

Human-in-the-Loop

LangGraph has built-in interrupt support. Pause execution at any node and wait for human input. The graph state is persisted, so you can resume days later. Use this for: approval workflows, ambiguity resolution, destructive action confirmation.

```

from langgraph.graph import StateGraph, END from typing
import TypedDict, Literal class AgentState(TypedDict):
messages: list task: str result: str
needs_clarification: bool def research_node(state:
AgentState) → AgentState: # Search and gather
information return {**state, "result": search_result}
def route(state: AgentState) → Literal["clarify",
"respond", END]: if state["needs_clarification"]:
return "clarify" if state["result"]: return "respond"
return END graph = StateGraph(AgentState)
graph.add_node("research", research_node)
graph.add_conditional_edges("research", route) app =
graph.compile(checkpointer=MemorySaver())

```

PRO TIP

LangGraph: typed state, pure function nodes, conditional routing, built-in human-in-the-loop. The right way to build production agents.

KEY LEARNINGS

- ✓ Nodes are pure functions: state_in → state_out
- ✓ Conditional edges: route based on what happened
- ✓ State is typed — no more mysterious agent failures
- ✓ interrupt() pauses for human approval and resumes seamlessly
- ✓ Compile your graph and visualize it with graph.get_graph().draw_mermaid()

Production AI Evaluation

The only thing worse than AI that doesn't work is AI that doesn't work and you don't know it. Build an evaluation system that catches regressions before users do.

LLM-as-Judge

Use GPT-4 to evaluate GPT-4o-mini's outputs. Define rubrics: 'Rate this answer 1-5 for accuracy, relevance, and safety. Return JSON.'

LLM judges correlate well with human judgments and scale infinitely. Build an automated eval pipeline that runs on every deployment.

Regression Testing

Build a golden dataset: 100 question-answer pairs where you know the correct answer. Run this on every model/prompt change. If your average score drops by more than 2%, reject the change. Treat AI evaluation like software testing — automate it.

A/B Testing in Production

Use feature flags to send 10% of traffic to the new prompt/model. Log all inputs and outputs. Run your eval metrics on both groups. Only promote the new version if metrics improve. Roll back instantly if they degrade.

```
import json from openai import OpenAI client = OpenAI()
def evaluate_response(question: str, answer: str,
```

```

context: str) → dict: result =
client.chat.completions.create( model="gpt-4o",
messages=[{"role": "user", "content": f"" Evaluate
this AI response on three criteria (1-5 each): 1.
Faithfulness: Is it supported by the context? 2.
Relevance: Does it answer the question? 3. Quality: Is
it clear and well-written? Question: {question}
Context: {context} Answer: {answer} Return JSON:
{"faithfulness": N, "relevance": N, "quality": N,
"reasoning": " ... "}}""}], response_format={"type":
"json_object"} ) return
json.loads(result.choices[0].message.content) # Run on
your golden dataset scores = [evaluate_response(q, a,
c) for q, a, c in golden_dataset] avg_faithfulness =
sum(s["faithfulness"] for s in scores) / len(scores)
print(f"Avg faithfulness: {avg_faithfulness:.2f}/5")

```

PRO TIP

Build a golden dataset. Automate LLM-as-judge evaluation. A/B test changes. Alert on regressions. Treat AI quality like software quality — measure it continuously.

KEY LEARNINGS

- ✓ LLM-as-judge scales infinitely and correlates with human judgment
- ✓ Golden dataset: 100 known Q&A pairs for regression testing
- ✓ A/B test every prompt/model change with feature flags
- ✓ Track cost per conversation alongside quality metrics
- ✓ Alert when evaluation scores drop below thresholds

Multimodal AI: Vision, PDFs & Screenshot Agents

GPT-4V and Claude Vision can read your screen, analyze images, and understand documents. The best AI engineers in 2025 build systems that work in the visual world. This chapter builds real multimodal agents.

Vision Models: What They Can Actually Do

Vision-language models (VLMs) accept images alongside text. GPT-4o, Claude 3, and Gemini Ultra all support this natively. The input is simple — base64-encoded image + text prompt — but the applications are profound. You can analyze UI screenshots, extract data from PDFs, compare product images, and build agents that look at your screen and take action.

The key insight: treat images as another input modality, not a special case. A multimodal RAG system stores image embeddings alongside text embeddings. This gives you retrieval that works on diagrams, screenshots, scanned forms, and charts — not just plain text documents.

```
import base64 from openai import OpenAI from pathlib
import Path client = OpenAI() def
analyze_image(image_path: str, question: str) -> str:
with open(image_path, "rb") as f: img_b64 =
base64.b64encode(f.read()).decode() ext =
Path(image_path).suffix.lower() mime = {".jpg": "image/
jpeg", ".png": "image/png"}.get(ext, "image/jpeg") resp
```

```

= client.chat.completions.create( model="gpt-4o",
messages=[{"role": "user", "content": [ {"type":
"image_url", "image_url": { "url": f"data:
{mime};base64,{img_b64}", "detail": "high" # high = 4x
tokens, better accuracy }}, {"type": "text", "text":
question} ]}], max_tokens=1500 ) return
resp.choices[0].message.content result =
analyze_image( "dashboard.png", "Extract all visible
metrics as JSON with labels and values." )
print(result)

```

PDF Screenshot Agents

PDFs with embedded images, charts, or handwritten content cannot be processed by standard text extraction. The solution: render each PDF page as an image, then use a VLM to extract data. This is dramatically more accurate for financial reports, medical records, and scanned contracts.

A screenshot agent takes this further — it literally takes a screenshot of your desktop, analyzes it, and decides what to do next. Tools like Playwright + GPT-4V can browse the web visually, fill forms, and extract data from sites that block scrapers.

```

import fitz # pip install pymupdf import base64 from
openai import OpenAI client = OpenAI() def
pdf_to_structured_data(pdf_path: str) -> list[dict]:
doc = fitz.open(pdf_path) results = [] for page_num,
page in enumerate(doc): mat = fitz.Matrix(2.0, 2.0) #
2x zoom for better quality pix =
page.get_pixmap(matrix=mat) b64 =
base64.b64encode(pix.tobytes("png")).decode() resp =
client.chat.completions.create( model="gpt-4o",
messages=[{"role": "user", "content": [ {"type":
"image_url", "image_url": { "url": f"data:image/
png;base64,{b64}", "detail": "high" }}, {"type":
"text", "text": "Extract ALL text, tables, and numbers.
Return JSON: {text, tables, numbers}" } ]}],
max_tokens=2000 ) results.append({"page": page_num + 1,

```

```
"content": resp.choices[0].message.content})) return  
results
```

Building a Visual RAG System

Multimodal RAG stores and retrieves both text and images. The trick is generating text captions for images so they are searchable, while keeping the original images for the LLM to analyze at query time.

Architecture: for each image, generate a caption via VLM → embed the caption → store (caption_embedding, image_path) → at query time, retrieve by caption similarity → pass original images to LLM for the final answer. This combines the speed of text search with the accuracy of full image analysis.

```
from openai import OpenAI import chromadb, base64  
client = OpenAI() col =  
chromadb.Client().get_or_create_collection("visual_rag")  
def index_image(image_path: str, doc_id: str): b64 =  
base64.b64encode(open(image_path,  
"rb").read()).decode() resp =  
client.chat.completions.create( model="gpt-4o",  
messages=[{"role": "user", "content": [ {"type":  
"image_url", "image_url": {"url": f"data:image/  
png;base64,{b64}"}}], {"type": "text", "text": "Describe  
for search indexing. Include all text, data,  
charts."} ]}] ) caption =  
resp.choices[0].message.content emb =  
client.embeddings.create(model="text-embedding-3-  
small", input=caption) col.add(documents=[caption],  
embeddings=[emb.data[0].embedding], ids=[doc_id],  
metadatas=[{"image_path": image_path}]) def  
visual_query(question: str) -> str: q_emb =  
client.embeddings.create(model="text-embedding-3-  
small", input=question) results =  
col.query(query_embeddings=[q_emb.data[0].embedding],  
n_results=3) images = [] for meta in  
results["metadatas"][0]: b64 =  
base64.b64encode(open(meta["image_path"],
```

```
"rb").read()).decode() images.append({"type":  
"image_url", "image_url": {"url": f"data:image/  
png;base64,{b64}"}}) images.append({"type": "text",  
"text": question}) resp =  
client.chat.completions.create(model="gpt-4o",  
messages=[{"role": "user", "content": images}]) return  
resp.choices[0].message.content
```

PRO TIP

Use detail='low' for quick classification tasks and detail='high' for data extraction from complex documents. The cost difference is 4x — choose wisely per use case.

KEY LEARNINGS

- ✓ Vision-language models accept base64-encoded images inline with text prompts
- ✓ Render PDFs to images with PyMuPDF for accurate extraction of charts, tables, and handwriting
- ✓ Multimodal RAG indexes image captions for fast search but retrieves original images for the LLM
- ✓ Screenshot agents combine Playwright browser control with VLM analysis for visual web automation
- ✓ Use detail=high for data extraction tasks and detail=low for classification to control cost

Shipping AI SaaS Products

Building an AI feature is 20% of the work. Shipping it as a product people pay for, trust, and keep using — that is the other 80%. This chapter is the engineering guide to going from working prototype to revenue-generating SaaS.

The AI SaaS Architecture That Scales

Most AI SaaS apps fail not because the AI is bad, but because the infrastructure breaks under load. The right architecture: API gateway → async job queue → worker pool → results cache. This decouples the slow (LLM inference) from the fast (HTTP responses), lets you scale workers independently, and gives users real-time progress updates.

Key decisions: (1) Synchronous vs async — anything over 3 seconds should be async with a job ID. (2) Streaming — use Server-Sent Events for real-time token streaming. (3) Caching — cache embeddings aggressively, cache LLM responses for identical inputs. (4) Rate limiting — per-user, per-tier limits protect you from runaway costs.

```
# FastAPI + Redis async job queue pattern from fastapi
import FastAPI, BackgroundTasks from redis import Redis
import uuid, json app = FastAPI() redis =
Redis(host="localhost", decode_responses=True)
@app.post("/analyze") async def start_analysis(content:
str, bg: BackgroundTasks): job_id = str(uuid.uuid4())
redis.setex(f"job:{job_id}", 3600,
```

```

json.dumps({"status": "queued", "result": None}))
bg.add_task(run_analysis, job_id, content) return
{"job_id": job_id, "status": "queued"} @app.get("/jobs/
{job_id}") async def get_job(job_id: str): data =
redis.get(f"job:{job_id}") return json.loads(data) if
data else {"error": "Job not found"} async def
run_analysis(job_id: str, content: str):
redis.setex(f"job:{job_id}", 3600,
json.dumps({"status": "processing"})) try: result =
await call_llm(content) redis.setex(f"job:{job_id}",
3600, json.dumps({"status": "done", "result": result}))
except Exception as e: redis.setex(f"job:{job_id}",
3600, json.dumps({"status": "failed", "error":
str(e)}))

```

Cost Control and Usage Billing

AI SaaS is brutally different from regular SaaS: your cost of goods is variable. Every GPT-4 call costs real money. Without cost controls, a single viral moment can bankrupt you before you notice.

Must-haves: (1) Per-request token counting before calling the LLM. (2) Monthly usage caps per user tier. (3) Cost alerts when a user's spend crosses a threshold. (4) Credit-based pricing where users buy credits upfront. (5) Model routing — use GPT-4o-mini for simple tasks, cutting costs by 97% for easy queries.

```

import tiktoken from openai import OpenAI client =
OpenAI() encoder =
tiktoken.encoding_for_model("gpt-4o") COSTS =
{ "gpt-4o": {"input": 0.005, "output": 0.015}, "gpt-4o-
mini": {"input": 0.00015, "output": 0.0006}, } def
count_tokens(text: str) -> int: return
len(encoder.encode(text)) def estimate_cost(prompt:
str, model="gpt-4o", output_tokens=500) -> float:
in_tok = count_tokens(prompt) c = COSTS[model] return
(in_tok * c["input"] + output_tokens * c["output"]) /
1000 def route_model(prompt: str) -> str: needs_deep =
any(kw in prompt.lower() for kw in ["analyze",

```

```

"compare", "design", "evaluate"]]) return "gpt-4o" if
needs_deep else "gpt-4o-mini" def
check_user_credits(user_id: str, cost: float) -> bool:
balance = get_user_balance(user_id) # from your DB if
balance < cost: raise ValueError(f"Insufficient
credits: need ${cost:.4f}, have ${balance:.4f}")
deduct_credits(user_id, cost) return True # Example
usage prompt = "Summarize this 5-page report..." model
= route_model(prompt) # -> gpt-4o-mini (97% cheaper)
cost = estimate_cost(prompt, model)
check_user_credits("user_123", cost) result =
client.chat.completions.create( model=model,
messages=[{"role": "user", "content": prompt}] )

```

Reliability: Retries, Fallbacks, and Observability

LLM APIs go down. They timeout. They return garbage. Production AI apps need automatic retries with exponential backoff, fallback models, and observability that tells you exactly why something failed.

Use LangSmith, Helicone, or Langfuse to trace every LLM call. You should be able to replay any user session, see exactly what prompt was sent, what came back, how long it took, and what it cost.

Without observability, debugging production issues is guesswork.

```

import time, random from openai import OpenAI,
RateLimitError, APIError client = OpenAI() def
llm_with_fallback( messages: list, primary_model: str =
"gpt-4o", fallback_model: str = "gpt-4o-mini",
max_retries: int = 3 ) -> str: for attempt in
range(max_retries): model = primary_model if attempt <
2 else fallback_model try: resp =
client.chat.completions.create( model=model,
messages=messages, timeout=30 ) return
resp.choices[0].message.content except RateLimitError:
wait = (2 ** attempt) + random.uniform(0, 1)
time.sleep(wait) except APIError: if attempt ==
max_retries - 1: raise time.sleep(2 ** attempt) raise

```

```
Exception("All retries exhausted") # Structured logging
for observability import logging, json logger =
logging.getLogger("ai.calls") def logged_call(messages:
list, model: str, user_id: str) -> str: start =
time.time() result = llm_with_fallback(messages, model)
logger.info(json.dumps({ "user_id": user_id, "model":
model, "latency_ms": int((time.time() - start) *
1000), })) return result
```

PRO TIP

Set a hard monthly spend limit in your OpenAI or Anthropic dashboard. This is your last line of defense. Set it at 2x expected monthly spend and adjust as you grow.

KEY LEARNINGS

- ✓ Use async job queues (Redis + background tasks) for any LLM call that takes over 3 seconds
- ✓ Implement per-user token budgets and monthly spend caps before onboarding your first paying user
- ✓ Route simple tasks to GPT-4o-mini (97% cheaper) and complex tasks to GPT-4o using a classifier
- ✓ Retry with exponential backoff on rate limits and fall back to cheaper models on persistent failures
- ✓ Use LangSmith or Helicone to trace every LLM call in production — observability is non-negotiable

Self-Hosted LLMs with Ollama & vLLM

Not every AI workload belongs in OpenAI's cloud. Private data, compliance requirements, latency constraints, and cost at scale all push toward self-hosted models. Ollama makes it trivial for development; vLLM makes it production-grade.

Ollama: Local LLMs in 5 Minutes

Ollama is the fastest way to run LLMs locally. One command to install, one to pull a model, one to start an OpenAI-compatible API server. It handles quantization, GPU/CPU fallback, and model management automatically.

Best models for local use: Llama 3.1 8B (8GB RAM, great quality), Mistral 7B (fast, good at code), Phi-3 Mini (very fast, surprisingly capable), Qwen2.5-Coder (best local coding model). Quantization sweet spot: Q4_K_M gives 4-bit quality with good accuracy retention.

```
# Install: curl https://ollama.ai/install.sh | sh #
Pull: ollama pull llama3.1:8b # Serve: ollama serve #
runs on localhost:11434 # Drop-in replacement for
OpenAI SDK from openai import OpenAI client =
OpenAI( base_url="http://localhost:11434/v1",
api_key="ollama" # required by SDK but ignored by
Ollama ) def local_chat(prompt: str, model: str =
"llama3.1:8b") -> str: resp =
```

```

client.chat.completions.create( model=model,
messages=[{"role": "user", "content": prompt}] ) return
resp.choices[0].message.content # Streaming def
local_stream(prompt: str): stream =
client.chat.completions.create( model="llama3.1:8b",
messages=[{"role": "user", "content": prompt}],
stream=True ) for chunk in stream: if
chunk.choices[0].delta.content:
print(chunk.choices[0].delta.content, end="",
flush=True) # LangChain integration from
langchain_ollama import ChatOllama llm =
ChatOllama(model="llama3.1:8b", temperature=0) result =
llm.invoke("Explain RAG in 3 sentences")
print(result.content)

```

vLLM: Production GPU Inference

Ollama is great for development. For production serving, vLLM is the standard. It uses PagedAttention to serve 20-30x more requests per GPU than naive inference, supports tensor parallelism, and exposes an OpenAI-compatible API.

Deploy vLLM on a single A100 80GB and serve Llama 3.1 70B at ~200 tokens/second — handling hundreds of concurrent users.

Cost crossover vs OpenAI GPT-4o: roughly 200K output tokens per day.

```

# Install: pip install vllm # Start: python -m
vllm.entrypoints.openai.api_server \ # --model meta-
llama/Meta-Llama-3.1-8B-Instruct \ # --port 8000 --
tensor-parallel-size 1 from openai import OpenAI import
asyncio, aiohttp client = OpenAI(base_url="http://
localhost:8000/v1", api_key="token-abc123") # Batch
inference – vLLM handles concurrent requests
efficiently async def batch_generate(prompts:
list[str]) -> list[str]: async with
aiohttp.ClientSession() as session: tasks =
[ session.post("http://localhost:8000/v1/chat/
completions", json={ "model": "meta-llama/Meta-

```

```
Llama-3.1-8B-Instruct", "messages": [{"role": "user",
"content": p}], "max_tokens": 512 }) for p in prompts ]
responses = await asyncio.gather(*tasks) results = []
for r in responses: data = await r.json()
results.append(data["choices"][0]["message"]
["content"]) return results # Process 100 documents in
parallel docs = ["Summarize: " + d for d in
load_documents()] outputs =
asyncio.run(batch_generate(docs))
```

Smart Model Router: API vs Local

Self-hosting wins when: data cannot leave your infrastructure, you have high volume (over 1M tokens per day), you have fine-tuned models, or you need sub-100ms latency. API wins when prototyping, needing frontier model quality, or having low volume.

Build a smart router that classifies query complexity and sends simple queries to local models and complex ones to cloud APIs. This approach cuts costs by 60-80% with minimal quality loss.

```
from openai import OpenAI from enum import Enum class
ModelTier(Enum): LOCAL_FAST = ("ollama", "phi3:mini") #
< 100ms, trivial tasks LOCAL_QUALITY = ("ollama",
"llama3.1:8b") # ~ 500ms, most tasks CLOUD_CHEAP =
("openai", "gpt-4o-mini") # cloud, medium tasks
CLOUD_BEST = ("openai", "gpt-4o") # cloud, complex
tasks def classify(prompt: str) -> ModelTier: has_code
= any(kw in prompt for kw in ["def ", "class ", "import
"]) is_complex = any(kw in prompt.lower() for kw in
["analyze", "compare", "design", "evaluate"]) is_long =
len(prompt.split()) > 500 if is_complex or has_code:
return ModelTier.CLOUD_BEST if is_long: return
ModelTier.LOCAL_QUALITY return ModelTier.LOCAL_FAST def
smart_complete(prompt: str) -> str: tier =
classify(prompt) provider, model = tier.value base_url
= "http://localhost:11434/v1" if provider == "ollama"
else None api_key = "ollama" if provider == "ollama"
else None client = OpenAI(base_url=base_url,
```

```
api_key=api_key) resp =  
client.chat.completions.create( model=model,  
messages=[{"role": "user", "content": prompt}] ) return  
resp.choices[0].message.content
```

PRO TIP

Llama 3.1 8B at Q4_K_M on a single RTX 3090 outperforms GPT-3.5 on most tasks. You do not always need a 70B model — benchmark your actual workload before provisioning expensive hardware.

KEY LEARNINGS

- ✓ Ollama runs LLMs locally with one command and exposes an OpenAI-compatible API on port 11434
- ✓ vLLM uses PagedAttention for 20-30x higher GPU throughput than naive single-request inference
- ✓ Q4_K_M quantization gives the best quality-to-size ratio for local model deployment
- ✓ Self-hosting wins for data privacy, high-volume workloads, fine-tuned models, and low latency needs
- ✓ A model router that sends simple queries local and complex ones to cloud APIs cuts costs 60-80%

The AI Architect's Playbook

Anyone can glue together an LLM call. An AI architect designs systems that are reliable at scale, cost-efficient, and maintainable. This chapter is the senior engineer's guide to AI system design patterns, guardrails, and evaluation.

Core AI Design Patterns

The patterns that solve 90% of AI design problems: Pipeline chain of transformations where each step refines the output (preprocess > retrieve > augment > generate > postprocess). Each step is independently testable and replaceable. Map-Reduce split large documents into chunks, process in parallel, combine results. Ideal for summarizing long documents or batch-processing large datasets efficiently.

Pattern Pattern

Reflection Pattern: generate, critique, then revise. The LLM critiques its own output against specific criteria, then revises. Improves quality by 15-30% on structured tasks without any extra training. This simple loop turns a single LLM call into a self-improving system.

```
from openai import OpenAI client = OpenAI() def
generate_with_reflection(task: str, criteria:
list[str]) -> str: criteria_str = "\n".join(f"- {c}"
for c in criteria) # Step 1: First draft draft =
client.chat.completions.create( model="gpt-4o",
```

```

messages=[{"role": "user", "content": f"Complete this
task:\n{task}"}] ).choices[0].message.content # Step 2:
Critique against criteria critique =
client.chat.completions.create( model="gpt-4o",
messages=[{"role": "user", "content": ( f"Review this
draft against:\n{criteria_str}\n\n" f"Draft:\n{draft}
\n\nFor each criterion: PASS or FAIL with
reason." )}] ).choices[0].message.content # Step 3:
Improved version final =
client.chat.completions.create( model="gpt-4o",
messages=[ {"role": "user", "content": f"Task:
{task}"}, {"role": "assistant", "content": draft},
{"role": "user", "content": f"Critique:\n{critique}
\n\nProduce improved
version:"} ] ).choices[0].message.content return final
result = generate_with_reflection( task="Write a
technical summary of RAG systems", criteria=["Includes
code example", "Under 500 words", "No unexplained
jargon"] )

```

Guardrails: PII Detection and Prompt Injection

Production AI systems need multiple layers of defense: input validation, PII detection, prompt injection blocking, and output validation. Users will send private data, attempt jailbreaks, and trigger edge cases you need to handle all of these automatically and silently.

cases

Architecture: Input guard (PII redaction, injection check, token limit) to LLM to Output guard (harmful content check, format validation) to Response. Each guard is a separate, independently improvable module. This layered design lets you add new guards without touching the core LLM logic.

```

import re from openai import OpenAI client = OpenAI()
PII_PATTERNS = { "ssn": r"\b\d{3}-\d{2}-\d{4}\b",
"credit_card": r"\b\d{4}[\s-]?\d{4}[\s-]?\d{4}[\s-]?"

```

```

\d{4}\b", "email": r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b", } def redact_pii(text: str) -> tuple[str, list]: found = [] for kind, pattern in PII_PATTERNS.items(): if re.search(pattern, text): found.append(kind) text = re.sub(pattern, f"[{kind.upper()}_REDACTED]", text) return text, found def is_injection(text: str) -> bool: signals = ["ignore previous", "forget your", "you are now", "new instructions:"] return any(s in text.lower() for s in signals) def safe_llm_call(user_input: str, system_prompt: str) -> dict: if is_injection(user_input): return {"status": "blocked", "reason": "potential prompt injection"} clean, redacted = redact_pii(user_input) resp = client.chat.completions.create( model="gpt-4o", messages=[ {"role": "system", "content": system_prompt}, {"role": "user", "content": clean} ] ) return {"status": "ok", "response": resp.choices[0].message.content, "pii_removed": redacted}

```

Automated Evaluation at Scale

How do you know your AI system is actually good? Not with with automated evaluation pipelines. The three metrics that matter: accuracy (correct answers), latency (fast enough), and cost (sustainable at scale).

vibes

Build an eval suite with 100-500 representative inputs and expected outputs. Run it after every significant change. Use LLM-as-judge for open-ended it correlates with human judgment at 80%+ agreement on most task domains. Treat your eval score like a test suite: any regression below a threshold blocks the deploy.

quality

```

import json, time from openai import OpenAI from dataclasses import dataclass client = OpenAI()

```

```

@dataclass class EvalResult: question: str expected:
str actual: str score: float latency_ms: int def
llm_judge(question: str, expected: str, actual: str) ->
float: resp =
client.chat.completions.create( model="gpt-4o-mini",
messages=[{"role": "user", "content": ( f"Score 0.0-1.0
on accuracy and completeness.\n\n" f"Question:
{question}\nExpected: {expected}\nActual: {actual}\n\n"
"Return JSON: {score: 0.0}" )}],
response_format={"type": "json_object"} ) return
json.loads(resp.choices[0].message.content).get("score",
0.0) def run_eval(test_cases: list[dict], system_fn) ->
dict: results = [] for tc in test_cases: start =
time.time() actual = system_fn(tc["question"]) ms =
int((time.time() - start) * 1000) score =
llm_judge(tc["question"], tc["expected"], actual)
results.append(EvalResult(tc["question"],
tc["expected"], actual, score, ms)) return
{ "avg_score": round(sum(r.score for r in results) /
len(results), 3), "avg_latency_ms":
round(sum(r.latency_ms for r in results) /
len(results)), "pass_rate": round(sum(1 for r in
results if r.score >= 0.7) / len(results), 3), }

```

PRO TIP

Build your eval suite before you build your system. Defining 20 test cases upfront forces clarity on requirements and gives you an objective measure of progress from day one.

KEY LEARNINGS

- ✓ Pipeline, Map-Reduce, and Reflection patterns solve 90% of AI system design challenges
- ✓ Layer input guards (PII redaction, injection detection) before every LLM call in production
- ✓ Build an automated eval suite with 100+ test cases and run it after every significant code change
- ✓ LLM-as-judge achieves 80%+ agreement with human evaluators for open-ended quality assessment
- ✓ Track accuracy, latency, and cost as your three primary AI system health metrics in dashboards

DSPy: Programmatic Prompt Optimization

Hand-written prompts are brittle. DSPy treats prompts as parameters to be optimized — not strings to tweak. It is the difference between gradient descent and manual weight adjustment. This chapter teaches you to build AI systems that improve themselves.

Why DSPy Exists: Beyond Manual Prompting

The problem with traditional prompting: write a prompt, it works on your test cases, fails on edge cases, patch the prompt, something else breaks. Whack-a-mole. DSPy solves this by separating the signature (what goes in and out) from the optimization (how the prompt gets there).

In DSPy you define a Signature — the input/output contract — and a Program — the computation graph. The optimizer searches for the best prompts, few-shot examples, and chain-of-thought strategies to maximize your metric. You stop writing prompts. You write evaluators.

```
import dspy # Configure the LLM lm = dspy.LM("openai/
gpt-4o-mini", temperature=0.7) dspy.configure(lm=lm) #
Signature: declare what goes in and what comes out
class QuestionAnswer(dspy.Signature): '''Answer with a
concise, accurate response.''' question: str =
dspy.InputField() context: str =
dspy.InputField(desc="Relevant background information")
```

```

answer: str = dspy.OutputField(desc="Concise answer,
1-3 sentences") # Program: composition of DSPy modules
class RAGProgram(dspy.Module): def __init__(self):
self.retrieve = dspy.Retrieve(k=3) self.answer =
dspy.ChainOfThought(QuestionAnswer) def forward(self,
question: str) -> dspy.Prediction: passages =
self.retrieve(question).passages context_str =
"\n".join(passages) return
self.answer(question=question, context=context_str) rag
= RAGProgram() result = rag(question="What is fine-
tuning and when should I use it?") print(result.answer)

```

Optimizing With MIPROv2

DSPy's MIPROv2 optimizer automatically searches for better prompts. It generates candidate instructions from training examples, selects few-shot examples that maximize your metric, evaluates candidates on a validation set, and returns the optimized program.

Result: programs that outperform hand-written prompts by 10-40% on complex tasks. The key is defining a good metric — the optimizer is only as good as what you tell it to optimize for. Use LLM-as-judge for open-ended quality.

```

import dspy from dspy.teleprompt import MIPROv2 # Your
metric — this is what the optimizer maximizes def
semantic_metric(example, pred, trace=None) -> float:
judge = dspy.Predict("question, expected, predicted ->
score: float") result =
judge( question=example.question,
expected=example.answer, predicted=pred.answer ) try:
return min(max(float(result.score), 0.0), 1.0) except
(ValueError, AttributeError): return 0.0 # Training
data: minimum 30-50 examples trainset =
[ dspy.Example( question="What is RAG?", context="RAG
stands for Retrieval-Augmented Generation...",
answer="RAG combines document retrieval with LLM
generation for accurate, grounded
responses." ).with_inputs("question", "context"), # ...

```

```

more examples ] # Optimize program = RAGProgram()
optimizer = MIPROv2(metric=semantic_metric,
auto="medium") optimized = optimizer.compile( program,
trainset=trainset, max_bootstrapped_demos=4,
num_trials=30, ) # Save and reload without re-running
optimization optimized.save("optimized_rag.json")
loaded = RAGProgram().load("optimized_rag.json")

```

DSPy Assertions: Constrained Generation

DSPy Assertions add runtime constraints that must be satisfied. If an assertion fails, DSPy automatically retries the module with the constraint violation injected as context — up to `max_backtracks` times. This gives you reliable structured outputs without brittle JSON parsing loops.

Assertions are enforced both at runtime and during optimization — the optimizer finds prompts that naturally satisfy your constraints, reducing runtime failures in production.

```

import dspy class StructuredExtractor(dspy.Module): def
__init__(self): self.extract =
dspy.ChainOfThought( "document -> entities: list,
sentiment: str, topics: list" ) def forward(self,
document: str) -> dspy.Prediction: pred =
self.extract(document=document)
dspy.Assert( isinstance(pred.entities, list) and
len(pred.entities) > 0, "Must extract at least one
named entity from the document" )
dspy.Assert( pred.sentiment in ["positive", "negative",
"neutral", "mixed"], f"Sentiment must be one of:
positive, negative, neutral, mixed" )
dspy.Assert( isinstance(pred.topics, list) and 1 <=
len(pred.topics) <= 5, "Must extract between 1 and 5
main topics" ) return pred # Enable retry-on-assertion-
failure dspy.configure(lm=dspy.LM("openai/gpt-4o"),
max_errors=3) extractor =
dspy.assert_transform_module(StructuredExtractor(),
max_backtracks=3) result = extractor("Apple released

```

```
new iPhone models today with improved cameras...")
print(result.entities, result.sentiment, result.topics)
```

PRO TIP

Start with 50 training examples and 20 validation examples. DSPy achieves meaningful optimization with small datasets. Diminishing returns kick in after roughly 200 examples.

KEY LEARNINGS

- ✓ DSPy separates what you want (Signature) from how to get it (optimization), replacing brittle prompts
- ✓ MIPROv2 automatically discovers better prompts, instructions, and few-shot examples for your metric
- ✓ DSPy Assertions add runtime constraints that trigger automatic retry with corrective context
- ✓ Define a semantic LLM-as-judge metric for open-ended tasks rather than exact-match comparison
- ✓ Save and load optimized programs to avoid expensive re-optimization on every deployment

GraphRAG: Knowledge Graphs for Complex Reasoning

Standard RAG retrieves isolated chunks. GraphRAG understands relationships — who worked with whom, what caused what, how concepts connect. For multi-hop questions spanning multiple documents, GraphRAG is the upgrade standard RAG cannot provide.

Why Standard RAG Fails Multi-Hop Questions

Standard RAG fails on multi-hop questions that require connecting information across sources. Which AI researchers who worked at Google also contributed to transformer architecture? This requires knowing who worked at Google, who contributed to transformers, and finding the overlap. Chunk retrieval cannot do this reliably.

GraphRAG solves this by building a knowledge graph: entities (people, organizations, concepts) as nodes, relationships (worked-at, contributed-to, caused-by) as edges. Complex questions become graph traversal problems solved with precision.

```
from neo4j import GraphDatabase from openai import
OpenAI import json client = OpenAI() driver =
GraphDatabase.driver("bolt://localhost:7687",
auth=("neo4j", "password")) def
extract_graph_data(text: str) -> dict: resp =
client.chat.completions.create( model="gpt-4o",
messages=[{"role": "user", "content": ( "Extract
```

```

entities and relationships from this text.\n" "Return
JSON: {\\"entities\\": [{\\"id\\": \\"e1\\", \\"name\\": \\"...
\\", \\"type\\": \\"Person|Org|Concept\\"}], "
\\"relationships\\": [{\\"from\\": \\"e1\\", \\"to\\": \\"e2\\",
\\"type\\": \\"WORKED_AT|CREATED|COLLABORATED\\"}]]\n\n"
f"Text: {text}" )]], response_format={"type":
"json_object"} ) return
json.loads(resp.choices[0].message.content) def
build_graph(documents: list[str]): with
driver.session() as session: for doc in documents: data
= extract_graph_data(doc) for e in data.get("entities",
[]): session.run("MERGE (n:Entity {id: $id}) SET
n.name=$name, n.type=$type", id=e["id"],
name=e["name"], type=e["type"]) for r in
data.get("relationships", []): rel = r["type"]
session.run( f"MATCH (a:Entity {{id: $fid}}), (b:Entity
{{id: $tid}}) MERGE (a)-[:{rel}]->(b)", fid=r["from"],
tid=r["to"] )

```

Graph-Augmented Question Answering

GraphRAG query pipeline: (1) Parse question to identify entities. (2) Query the knowledge graph to find relevant subgraphs up to 2 hops. (3) Convert the subgraph to natural language context. (4) Pass to LLM for the final answer.

The key innovation: graph traversal gives you precise, relationship-aware retrieval. Instead of finding chunks that mention transformers, you get find all researchers connected to the transformer paper through AUTHORED edges within 2 hops. This is 10x more precise for complex queries.

```

def graph_rag_query(question: str) -> str: # Step 1:
Extract entities from question parse =
client.chat.completions.create( model="gpt-4o-mini",
messages=[{"role": "user", "content": ( f"Extract named
entities from: {question}\n" "Return JSON:
{\\"entities\\": [\\"name1\\", \\"name2\\"}]" )]],
response_format={"type": "json_object"} ) names =

```

```

json.loads(parse.choices[0].message.content).get("entities"
[]) # Step 2: Traverse graph (up to 2 hops) with
driver.session() as session: cypher = ( "MATCH
(e:Entity)-[r*1..2]-(n:Entity) " "WHERE e.name IN
$names " "RETURN e.name AS src, " "[x IN r | type(x)]
AS rels, " "n.name AS tgt, n.type AS tgt_type " "LIMIT
50" ) rows = session.run(cypher, names=names) graph_ctx
= "\n".join( f"{row['src']} --{'-'
>'.join(row['rels'])}--> {row['tgt']}"
({row['tgt_type']})" for row in rows ) # Step 3:
Generate answer resp =
client.chat.completions.create( model="gpt-4o",
messages=[{"role": "user", "content": ( f"Use this
knowledge graph context to answer precisely.\n\n"
f"Graph:\n{graph_ctx}\n\nQuestion: {question}" )}] )
return resp.choices[0].message.content

```

Microsoft GraphRAG: Community Summaries

Microsoft's open-source GraphRAG (2024) adds community detection. After building the knowledge graph it runs the Leiden algorithm to find clusters of closely related entities, then generates a summary for each cluster. These community summaries enable global queries — questions about themes and patterns across an entire corpus.

Two query modes: Local (specific factual queries — traverse graph + retrieve text chunks) and Global (thematic queries — synthesize community summaries). For corpora over 100 documents, this distinction is crucial.

```

# pip install graphrag # # Initialize: graphrag init --
root ./mydata # Index: graphrag index --root ./mydata #
(buils entity graph, relationships, and community
summaries automatically) # # CLI queries: # graphrag
query --root ./mydata --method local --query "Who
founded OpenAI?" # graphrag query --root ./mydata --
method global --query "What are the main AI themes?" #

```

```

Python SDK from
graphrag.query.structured_search.local_search.search
import LocalSearch from
graphrag.query.llm.oai.chat_openai import ChatOpenAI
import tiktoken, asyncio llm =
ChatOpenAI(api_key="your-key", model="gpt-4o",
api_type="openai") local_search = LocalSearch( llm=llm,
context_builder=context_builder, # built from indexed
graph
token_encoder=tiktoken.get_encoding("cl100k_base"),
response_type="multiple paragraphs", ) async def
query(q: str, mode: str = "local") -> str: search =
local_search if mode == "local" else global_search
result = await search.asearch(q) return result.response
# Local: specific facts print(asyncio.run(query("What
companies did Sam Altman found?", "local"))) # Global:
themes and patterns print(asyncio.run(query("What are
the key trends in LLM research?", "global")))

```

PRO TIP

Start with 50-100 documents to validate your graph schema before scaling. The entity types and relationship types you define now determine what questions you can answer later.

KEY LEARNINGS

- ✓ GraphRAG builds a knowledge graph of entities and relationships enabling precise multi-hop reasoning
- ✓ Neo4j Cypher queries retrieve relationship-aware context that chunk-based retrieval cannot provide
- ✓ Microsoft GraphRAG adds Leiden community detection for thematic global queries across entire corpora
- ✓ Local search answers specific factual queries; Global search answers thematic pattern questions
- ✓ Entity extraction quality determines graph quality — use GPT-4o for extraction, mini for queries

AI Memory Systems: Episodic, Semantic & Procedural

ChatGPT forgets you the moment the conversation ends. Production AI assistants remember your preferences, past interactions, and learned patterns. This chapter builds the three types of memory that make AI systems feel genuinely intelligent.

The Three Types of AI Memory

Human memory has three types — and effective AI systems need all three:

Episodic Memory captures specific past events. Last Tuesday you asked me to analyze the Q3 report. Stored as timestamped conversation summaries with semantic search. Enables continuity across sessions.

Semantic Memory stores facts. You prefer Python over JavaScript. Your company uses AWS. You are building a RAG system. Stored as a structured knowledge base and updated as new facts emerge.

Procedural Memory encodes how to do things. When you ask for code, you want type hints and tests. When summarizing, you want bullet points. Stored as preference rules, learned from corrections and feedback.

```

from datetime import datetime from openai import OpenAI
import chromadb, json client = OpenAI() chroma =
chromadb.Client() # Separate collections per memory
type episodic =
chroma.get_or_create_collection("episodic_memory")
semantic =
chroma.get_or_create_collection("semantic_memory")
procedural =
chroma.get_or_create_collection("procedural_memory")
def embed(text: str) -> list[float]: return
client.embeddings.create( model="text-embedding-3-
small", input=text ).data[0].embedding def
store_episode(user_id: str, turns: list[dict]): summary
= client.chat.completions.create( model="gpt-4o-mini",
messages=[{"role": "user", "content": f"Summarize in
2-3 sentences:
{json.dumps(turns)}"}] ).choices[0].message.content
episodic.add( documents=[summary],
embeddings=[embed(summary)], ids=[f"{user_id}
_{datetime.now().isoformat()}"], metadatas=[{"user_id":
user_id, "ts": datetime.now().isoformat()}] ) def
recall(user_id: str, query: str) -> dict: q =
embed(query) w = {"user_id": user_id} return
{ "episodes": episodic.query(query_embeddings=[q],
n_results=2, where=w)["documents"][0], "facts":
semantic.query(query_embeddings=[q], n_results=4,
where=w)["documents"][0], "preferences":
procedural.query(query_embeddings=[q], n_results=2,
where=w)["documents"][0], }

```

Semantic Memory: User Knowledge Base

Semantic memory stores facts about the user that persist forever. Every conversation is an opportunity to learn: You mentioned you work at Stripe. You prefer concise answers. Your tech stack is FastAPI and PostgreSQL.

The extraction pipeline: after each conversation, run an LLM pass to extract new facts. Deduplicate using embedding similarity (distance

below 0.15 means duplicate). Store only truly new or updated facts to keep the store clean.

```
def extract_and_store_facts(user_id: str, conversation:
list[dict]): extracted =
json.loads( client.chat.completions.create( model="gpt-4o-
mini", messages=[{"role": "user", "content": ( "Extract
concrete, explicitly stated facts about the user.\n"
"Return JSON: {\\"facts\\": [\\"fact1\\"], \\"preferences\\":
[\\"pref1\\"]}\n\n" f"Conversation:
{json.dumps(conversation)}" )}],
response_format={"type":
"json_object"} ).choices[0].message.content ) for fact
in extracted.get("facts", []): emb = embed(fact)
existing = semantic.query(query_embeddings=[emb],
n_results=1, where={"user_id": user_id}) if
existing["distances"][0] and existing["distances"][0]
[0] < 0.15: semantic.update(ids=[existing["ids"][0]
[0]], documents=[fact]) # update else:
semantic.add( documents=[fact], embeddings=[emb],
ids=[f"{user_id}_f_{datetime.now().timestamp()}"],
metadatas=[{"user_id": user_id}] ) for pref in
extracted.get("preferences", []):
procedural.add( documents=[pref],
embeddings=[embed(pref)], ids=[f"{user_id}
_p_{datetime.now().timestamp()}"],
metadatas=[{"user_id": user_id}] )
```

Memory-Augmented Chat in Production

A memory-augmented system retrieves context from all three memory stores before each response. The system prompt is dynamically constructed from user preferences, relevant facts, and related past conversations. This makes the AI feel like it genuinely knows you.

Context window management: always inject preferences (small), top 3 relevant facts, top 2 recent episodes. Update memories after each meaningful conversation, not every single message.

```
def memory_chat(user_id: str, message: str, history:
list) -> str: memories = recall(user_id, message) #
Build dynamic system prompt from all three memory types
parts = ["You are a helpful AI with persistent memory
of this user."] if memories["preferences"]:
parts.append("\nUser Preferences:\n" + "\n".join(f"-
{p}" for p in memories["preferences"])) if
memories["facts"]: parts.append("\nKnown Facts:\n" +
"\n".join(f"- {f}" for f in memories["facts"][:4])) if
memories["episodes"]: parts.append("\nRelevant Past
Conversations:\n" + "\n".join(f"- {e}" for e in
memories["episodes"][:2])) messages = [{"role":
"system", "content": "\n".join(parts)}] messages +=
history[-10:] messages += [{"role": "user", "content":
message}] reply =
client.chat.completions.create( model="gpt-4o",
messages=messages ).choices[0].message.content # Update
memories in background (non-blocking) updated = history
+ [ {"role": "user", "content": message}, {"role":
"assistant", "content": reply} ] store_episode(user_id,
updated[-6:]) extract_and_store_facts(user_id,
updated[-6:]) return reply
```

PRO TIP

Extract facts only after meaningful conversations of 5+ turns. Running extraction after every single message floods your semantic store with noise and increases retrieval cost.

KEY LEARNINGS

- ✓ Episodic memory stores past conversation summaries; semantic stores facts; procedural stores preferences
- ✓ Use separate vector collections for each memory type filtered by user_id metadata
- ✓ Deduplicate facts with embedding similarity (distance less than 0.15) before storing new entries
- ✓ Dynamically construct the system prompt by retrieving from all three memory stores per message
- ✓ Always inject preferences; use relevance scoring to select which facts and episodes fit the context window

MCP: Model Context Protocol

Anthropic's Model Context Protocol is becoming the USB standard for AI tool integration. Write one MCP server and any MCP-compatible client can use it — Claude Desktop, Cursor, and any future AI app. This is the protocol that makes AI agents composable.

What MCP Solves and How It Works

Before MCP, connecting an AI to your database required custom code in every application. MCP standardizes this: an MCP server exposes resources, tools, and prompts via JSON-RPC. Any MCP client connects and gets access to all capabilities automatically.

MCP servers expose three things: Resources — data that can be read (files, DB records, API responses). Tools — functions the AI can call (run SQL, send email, search). Prompts — reusable templates with parameters. Transport: stdio (local subprocess) or HTTP+SSE (remote).

```
# pip install mcp from mcp.server import Server from
mcp.server.stdio import stdio_server from mcp.types
import Resource, Tool, TextContent import asyncio, json
server = Server("my-data-server")
@server.list_resources() async def list_resources():
    return [Resource(uri="db://summary", name="DB Summary",
description="Database table overview",
mimeType="application/json")] @server.read_resource()
```

```

async def read_resource(uri: str): if uri == "db://
summary": return [TextContent(type="text",
text=json.dumps(get_db_summary()))] raise
ValueError(f"Unknown resource: {uri}")
@server.list_tools() async def list_tools(): return
[Tool( name="query_db", description="Run a read-only
SQL SELECT query. Returns up to 100 rows.",
inputSchema={"type": "object", "properties": {"sql":
{"type": "string"}}, "required": ["sql"]} )]
@server.call_tool() async def call_tool(name: str,
arguments: dict): if name == "query_db": result =
safe_query(arguments["sql"]) return
[TextContent(type="text", text=json.dumps(result,
default=str))] raise ValueError(f"Unknown tool:
{name}") async def main(): async with stdio_server() as
(read, write): await server.run(read, write)
asyncio.run(main())

```

Building a Safe Database MCP Server

A practical MCP server connects your business data to any AI assistant. Define resources for read-only data and tools for safe actions. Good MCP design lets your AI both answer questions and take actions.

Security first: validate all SQL to block writes, implement rate limiting, log every tool call. MCP servers run with the same permissions as your application — treat them like a public API because any connected AI client can call them.

```

import re, sqlite3, json from mcp.server import Server
from mcp.types import Tool, TextContent server =
Server("analytics-server") FORBIDDEN = ["insert",
"update", "delete", "drop", "create", "alter",
"truncate"] def safe_query(sql: str) -> dict: lower =
sql.lower().strip() for kw in FORBIDDEN: if
re.search(rf"\b{kw}\b", lower): raise
ValueError(f"Write operation not allowed: {kw}") conn =
sqlite3.connect("analytics.db") conn.row_factory =

```

```

sqlite3.Row try: cur = conn.execute(sql) rows =
[dict(r) for r in cur.fetchmany(100)] more =
cur.fetchone() is not None return {"rows": rows,
"count": len(rows), "truncated": more} finally:
conn.close() @server.list_tools() async def
list_tools(): return [Tool( name="query_analytics",
description="Query analytics DB. Read-only. Max 100
rows.", inputSchema={"type": "object", "properties":
{ "sql": {"type": "string"}, "reason": {"type":
"string", "description": "Why you need this data"} },
"required": ["sql", "reason"]} )] @server.call_tool()
async def call_tool(name: str, arguments: dict): if
name == "query_analytics": result =
safe_query(arguments["sql"]) return
[TextContent(type="text", text=json.dumps(result))]
raise ValueError(f"Unknown tool: {name}")

```

Connecting MCP to Claude Desktop and Cursor

Connecting your MCP server to Claude Desktop is a single JSON config file edit. Claude discovers your tools automatically and uses them in any conversation. Cursor uses the same protocol — your server works in both without any changes.

Claude manages the server lifecycle: starts it as a subprocess when needed, stops it when done. No registration, no API approval process — just edit the config and restart Claude Desktop.

```

// claude_desktop_config.json // macOS: ~/Library/
Application Support/Claude/
claude_desktop_config.json // Windows: %APPDATA%
\Claude\claude_desktop_config.json { "mcpServers":
{ "my-analytics": { "command": "python", "args": ["/
path/to/mcp_server.py"], "env": { "DB_PATH": "/path/to/
analytics.db" } }, "filesystem": { "command": "npx",
"args": ["-y", "@modelcontextprotocol/server-
filesystem", "/Users/you/Documents"] }, "github":
{ "command": "npx", "args": ["-y",
"@modelcontextprotocol/server-github"], "env":

```

```
{ "GITHUB_PERSONAL_ACCESS_TOKEN": "ghp_..." } } } } //
```

After saving: restart Claude Desktop. // A hammer icon (hammer) appears in the chat when MCP tools are active. // Test: "Query my analytics for total revenue this month" // Claude will automatically call query_analytics with the right SQL.

PRO TIP

Publish your MCP server on PyPI or npm and anyone with Claude Desktop or Cursor can use it with one config line. This is how the MCP ecosystem grows — build something useful and share it.

KEY LEARNINGS

- ✓ MCP standardizes AI tool integration: resources (read), tools (actions), and prompts (templates)
- ✓ MCP servers run as stdio subprocesses locally or HTTP+SSE endpoints for remote deployment
- ✓ A single JSON config file connects your MCP server to Claude Desktop, Cursor, and other clients
- ✓ Always validate SQL and block write operations in read-only database tool interfaces
- ✓ MCP servers work across all compatible clients without any client-specific integration code

Build a Cursor AI Clone

Cursor is a \$400M company built on a simple idea: AI that understands your codebase. This chapter reverse-engineers the core architecture and builds a working minimal version — codebase indexing, semantic code search, streaming answers, and inline edits.

Indexing Your Codebase With AST Parsing

Cursor's core innovation is codebase-aware AI. Regular ChatGPT knows Python; Cursor knows YOUR Python — your functions, conventions, and architecture. This requires parsing your codebase into a searchable vector store.

Indexing pipeline: walk file tree → skip gitignored files → parse into chunks at function and class boundaries using AST → embed each chunk → store in vector DB with file path and line numbers.

Chunking at semantic boundaries rather than fixed token counts gives dramatically better retrieval quality.

```
import ast from pathlib import Path from openai import
OpenAI import chromadb client = OpenAI() col =
chromadb.Client().get_or_create_collection("codebase")
def parse_python_chunks(filepath: str) -> list[dict]:
    chunks = [] try: source =
    Path(filepath).read_text(encoding="utf-8") tree =
    ast.parse(source) lines = source.splitlines() for node
    in ast.walk(tree): if isinstance(node,
    (ast.FunctionDef, ast.AsyncFunctionDef, ast.ClassDef)):
    code = "\n".join(lines[node.lineno - 1 :
```

```

node.end_lineno])) chunks.append({ "code": code, "name":
node.name, "kind": type(node).__name__.replace("Def",
""), "file": filepath, "start": node.lineno, "end":
node.end_lineno }) except (SyntaxError,
UnicodeDecodeError): pass return chunks def
index_codebase(root_dir: str): exts = {".py", ".ts",
".tsx", ".js", ".jsx", ".go"} chunks = [] for p in
Path(root_dir).rglob("*"): if p.suffix in exts and
".git" not in p.parts and "node_modules" not in
p.parts: chunks.extend(parse_python_chunks(str(p))) for
i in range(0, len(chunks), 100): batch =
chunks[i:i+100] texts = [f"# {c['file']} | {c['kind']}
{c['name']}\n{c['code']}" for c in batch] embs =
client.embeddings.create(model="text-embedding-3-
small", input=texts) col.add(documents=texts,
embeddings=[e.embedding for e in embs.data],
ids=[f"{c['file']}:{c['start']}" for c in batch],
metadatas=[{"file": c["file"], "name": c["name"]} for c
in batch]) print(f"Indexed {len(chunks)} code chunks")

```

Semantic Code Search and Streaming Answers

The query flow: embed the question → retrieve top-K relevant code chunks, prioritizing the current file → build a context-aware prompt → stream the response via Server-Sent Events.

Streaming makes the assistant feel instant even on long responses. The priority order for context: current file first, then globally similar code. This mirrors what a senior engineer would look at when answering the same question.

```

from fastapi import FastAPI from fastapi.responses
import StreamingResponse from openai import OpenAI
import chromadb, json app = FastAPI() client = OpenAI()
col =
chromadb.Client().get_or_create_collection("codebase")
def get_context(question: str, current_file: str =
None, n: int = 8) -> str: emb =
client.embeddings.create(model="text-embedding-3-

```

```

small", input=question).data[0].embedding docs = [] if
current_file: cur = col.query(query_embeddings=[emb],
n_results=3, where={"file": current_file})
docs.extend(cur["documents"][0]) glob =
col.query(query_embeddings=[emb], n_results=n)
docs.extend(glob["documents"][0]) return "\n\n---
\n\n".join(dict.fromkeys(docs)[:n]) # deduplicate, keep
order @app.post("/ask") async def ask(question: str,
current_file: str = None): context =
get_context(question, current_file) async def stream():
s = client.chat.completions.create( model="gpt-4o",
stream=True, messages=[ {"role": "system", "content":
( "Expert coding assistant. Codebase context:\n\n"
f"{context}\n\nBe precise. Show diffs for changes." )},
{"role": "user", "content": question} ] ) for chunk in
s: if chunk.choices[0].delta.content: yield f"data:
{json.dumps({'t': chunk.choices[0].delta.content})}
\n\n" yield "data: [DONE]\n\n" return
StreamingResponse(stream(), media_type="text/event-
stream")

```

Inline Edit: Select, Describe, Get a Diff

Cursor's killer feature is inline edits — select code, describe the change, get a precise diff. The edit prompt must be strict: return ONLY the modified code, no explanation, no markdown fences. Use temperature 0.1 for deterministic, precise edits.

After generation, compute a unified diff with difflib and display additions in green and deletions in red. This gives the user full visibility into exactly what changed before they accept.

```

import difflib from openai import OpenAI client =
OpenAI() def ai_edit(original: str, instruction: str,
lang: str = "python") -> dict: resp =
client.chat.completions.create( model="gpt-4o",
temperature=0.1, # deterministic edits
messages=[ {"role": "system", "content": ( f"You are a
{lang} code editor. " "Return ONLY the modified code. "

```

```
"No explanation. No markdown fences. No added
comments." )}, {"role": "user", "content":
( f"Original:\n{original}\n\n" f"Instruction:
{instruction}\n\n" "Modified code:" )} ] ) new_code =
resp.choices[0].message.content.strip() # Strip
accidental markdown fences if
new_code.startswith("`"): new_code =
"\n".join(new_code.split("\n")[1:-1]) diff =
list(difflib.unified_diff( original.splitlines(keepends=True),
new_code.splitlines(keepends=True),
fromfile="original", tofile="modified", n=3 )) return
{ "original": original, "modified": new_code, "diff":
"".join(diff), "additions": sum(1 for l in diff if
l.startswith("+") and not l.startswith("+++")),
"deletions": sum(1 for l in diff if l.startswith("-")
and not l.startswith("---")), }
```

PRO TIP

Use watchdog to detect file changes and re-index only modified files incrementally. Full re-indexing on every save is unusable on codebases with more than a few hundred files.

KEY LEARNINGS

- ✓ Index code at function and class boundaries with AST parsing for semantically meaningful chunks
- ✓ Prioritize the current file then globally similar code when filling the context window
- ✓ Stream responses via Server-Sent Events so users see tokens immediately during generation
- ✓ Use temperature=0.1 for inline edits to get deterministic, precise code modifications
- ✓ Compute unified diffs with difflib and display additions and deletions before the user accepts

CrewAI vs LangGraph vs AutoGen

Three frameworks for multi-agent AI systems. Each has a fundamentally different philosophy. Choosing wrong means rewriting your system in 3 months. This chapter gives you the honest comparison you need to pick correctly the first time.

CrewAI: Role-Based Agent Teams

CrewAI models teams of AI agents with explicit roles, goals, and backstories. Agents are assigned tasks, use tools, and collaborate in a defined sequence. It is the most human-intuitive framework — you think in terms of a researcher, a writer, and an editor.

Best for: content pipelines, research workflows, tasks with clear role separation, rapid prototyping. Not ideal for: dynamic workflows where the task structure is unknown upfront, complex state management, or production systems needing fine-grained control.

```
from crewai import Agent, Task, Crew, Process from
crewai_tools import SerperDevTool search =
SerperDevTool() researcher = Agent( role="Senior
Research Analyst", goal="Find and synthesize the latest
information on AI trends", backstory="Expert researcher
with 10 years finding signal in noise", tools=[search],
llm="gpt-4o", verbose=True ) writer =
Agent( role="Technical Writer", goal="Transform
research into clear, compelling content",
backstory="Former engineering lead turned technical
```

```
communicator", llm="gpt-4o" ) research_task =
Task( description="Research the latest AI agent
frameworks in 2025. " "Focus on production deployments
and real-world performance.",
expected_output="Comprehensive report with key findings
and citations", agent=researcher ) writing_task =
Task( description="Write a clear 800-word blog post
based on the research. " "Target audience: senior
developers.", expected_output="Published-ready blog
post with intro, 3 sections, and conclusion",
agent=writer, context=[research_task] # depends on
research output ) crew = Crew(agents=[researcher,
writer], tasks=[research_task, writing_task],
process=Process.sequential, verbose=True) result =
crew.kickoff() print(result.raw)
```

LangGraph: Stateful Graph Workflows

LangGraph models agents as nodes in a directed graph with state flowing through edges. It is the most flexible and production-grade option — used internally by LangChain and deployed in serious production systems. Every state transition is explicit, checkpointing is built-in, and human-in-the-loop is first class.

Best for: complex conditional workflows, human approval steps, applications needing reliable state persistence, any system where you must know exactly what happened at each step. Steeper learning curve but maximum control.

```
from langgraph.graph import StateGraph, END from
langchain_openai import ChatOpenAI from typing import
TypedDict, Annotated import operator llm =
ChatOpenAI(model="gpt-4o") class State(TypedDict):
messages: Annotated[list, operator.add] research: str
draft: str approved: bool revision_count: int def
research_node(s: State) -> dict: topic = s["messages"]
[-1]["content"] resp = llm.invoke([{"role": "user",
"content": f"Research thoroughly: {topic}"}]) return
{"research": resp.content} def write_node(s: State) ->
```

```
dict: resp = llm.invoke([{"role": "user", "content":
f"Write a blog post based on:\n{s['research']}"}])
return {"draft": resp.content, "revision_count":
s.get("revision_count", 0) + 1} def review_node(s:
State) -> dict: resp = llm.invoke([{"role": "user",
"content": f"Review. Say APPROVED or REVISION:
[feedback].\n\n{s['draft']}"}]) return {"approved":
"APPROVED" in resp.content} def route(s: State) -> str:
if s.get("approved") or s.get("revision_count", 0) >=
2: return "end" return "write" g = StateGraph(State)
g.add_node("research", research_node)
g.add_node("write", write_node) g.add_node("review",
review_node) g.set_entry_point("research")
g.add_edge("research", "write") g.add_edge("write",
"review") g.add_conditional_edges("review", route,
{"end": END, "write": "write"}) app = g.compile()
```

AutoGen: Conversational Multi-Agent Systems

AutoGen from Microsoft models agents as autonomous chatbots that communicate with each other. The key feature: agents have back-and-forth conversations to solve problems, with a human proxy that can intervene at any point.

Best for: complex coding tasks, mathematical reasoning, open-ended problems where the solution path is unknown, and tasks that benefit from multi-perspective debate.

Selection Guide: sequential content pipeline → CrewAI. Complex conditional workflow in production → LangGraph. Open-ended coding or reasoning task → AutoGen. Need human approval steps → LangGraph.

```
import autogen llm_config = {"config_list": [{"model":
"gpt-4o", "api_key": "YOUR_KEY"}]} # Human proxy – can
run code and ask for human input user_proxy =
autogen.UserProxyAgent( name="User",
human_input_mode="TERMINATE",
```

```

max_consecutive_auto_reply=10,
code_execution_config={"work_dir": "workspace",
"use_docker": False} ) engineer =
autogen.AssistantAgent( name="Engineer",
llm_config=llm_config, system_message=( "Expert
software engineer. Write clean, tested, production-
quality code. " "Say TERMINATE when the task is fully
complete." ) ) reviewer =
autogen.AssistantAgent( name="Reviewer",
llm_config=llm_config, system_message=( "Senior code
reviewer. Check for bugs, security issues, and style. "
"Say LGTM when the code is production-ready." ) ) #
GroupChat: LLM decides which agent speaks next
groupchat = autogen.GroupChat( agents=[user_proxy,
engineer, reviewer], messages=[], max_round=20,
speaker_selection_method="auto" ) manager =
autogen.GroupChatManager(groupchat=groupchat,
llm_config=llm_config)
user_proxy.initiate_chat( manager, message=( "Build a
FastAPI endpoint that validates CSV uploads " "and
returns summary statistics as JSON. Include pytest
tests." ) )

```

PRO TIP

LangGraph is the safe production default. CrewAI is great for rapid prototyping with defined roles. AutoGen shines for agentic coding tasks where the solution path is genuinely unknown.

KEY LEARNINGS

- ✓ CrewAI: role-based teams and sequential processes — best for defined pipelines with clear role separation
- ✓ LangGraph: explicit stateful graphs with checkpointing — best for complex conditional production workflows
- ✓ AutoGen: conversational multi-agent debate — best for open-ended coding and reasoning problems
- ✓ LangGraph has the best human-in-the-loop support with built-in interrupt and resume at any graph node
- ✓ All three frameworks support OpenAI-compatible LLMs — switching providers is a one-line configuration change

You're Ready to Build.

You've learned everything from your first API call to GraphRAG, fine-tuning, and autonomous agents. The only thing left to do is ship something.



Full Video Course: <https://youtu.be/VX29jXVsyAI>



AI Stack Picker: <https://ai-stack-picker-nine.vercel.app>



GitHub: <https://github.com/rohith547>